

COMP7710

Programming Fundamentals

This week's lecture
Algorithms and Complexity

Course Convenor
Bernardo Pereira Nunes

Guest lecturer
Alberto F. Martin



Australian
National
University

Agenda

- > Computational Complexity (Algorithm and Problem Complexity)
- > Algorithm (Time) Complexity
 - >> Big O notation
 - >> Most common rates of growth
 - >> Working out time complexity with three example problems
 - >>> Summing elements in a list
 - >>> Max difference among any two elements in a list
 - >>> Find greatest element up to
- > Problem Complexity
- > Exercises



Computational Complexity - Definition

- > When an algorithm solves a problem in a computer, it consumes computational resources
- > Examples of computational resources required to solve a problem in a computer
 - >> Time
 - >> Space (i.e., memory)
 - >> Energy, communication, file I/O traffic, etc.
- > **Computational complexity** is the study of how problem size affects resource consumption

Key question: *how resource consumption scales (grows) with increasing problem size?*



Computational Complexity – Algorithm and Problem complexity

> Two different dimensions of computational complexity:

>> **Algorithm Complexity**: given a problem, study of **a particular algorithm** that solves the problem. Algorithm complexity is the subject matter of a field in computer science known as *Analysis of Algorithms*

>> **Problem Complexity**: given a problem, study of **any possible algorithm** that solves the problem. Problem complexity is the subject matter of a field in computer science known as *Computational Complexity Theory*

> In this course we will just focus on the first dimension, i.e., Algorithm Complexity



Algorithm complexity – Characterizing problem size

- > We need to identify a parameter that characterizes problem size
- > Usually denoted as the lower-case letter n
- > Examples of what n may denote:
 - >> the number of elements in a list
 - >> the number of rows/columns in a matrix (i.e., array of arrays)
 - >> the number of samples in a data set



Algorithm complexity – Types of analysis

- > We need to study the algorithm to determine how resource consumption grows as a function of n (*the difficult part!*)
- > Depending on the algorithm at hand, **the value of the input**, not just its size, may influence resource consumption
- > This leads to different variants of complexity (**we will focus on the first two**):
 - >> **Worst-case** analysis considers input of size n that maximizes resource consumption
 - >> **Best-case** analysis considers input of size n that minimizes resource consumption
 - >> **Average-case** analysis considers all possible inputs of size n and averages their resource consumption
 - >> **Amortized analysis** considers a sequence of executions of the algorithm over an input of size n , averaging their resource consumption. Typically, useful for data structures that implement operations with internal state, and depending on their internal state, may require a varying amount of resource consumption (e.g., Hash Tables)



Big O notation (formal definition)

Mathematical notation that we use to denote computational complexity

Assume that we have a problem of size n and an algorithm that we know takes $g(n)$ resource units to solve the problem

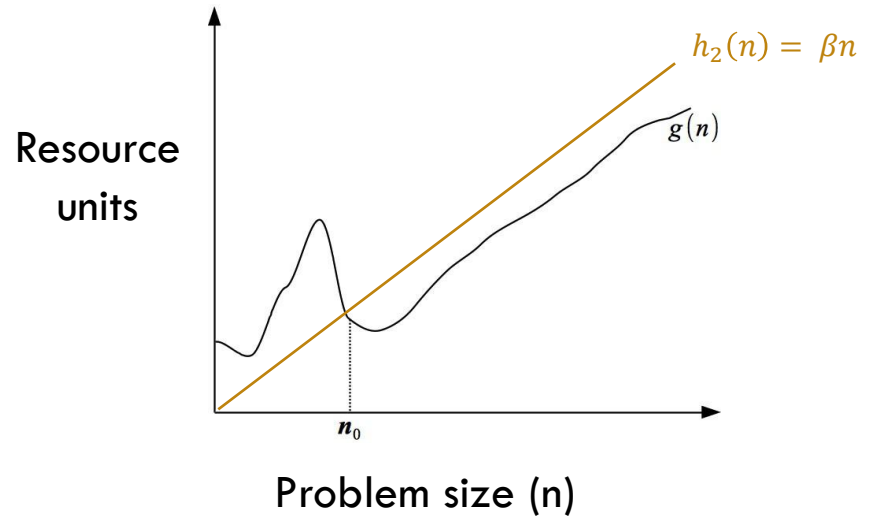
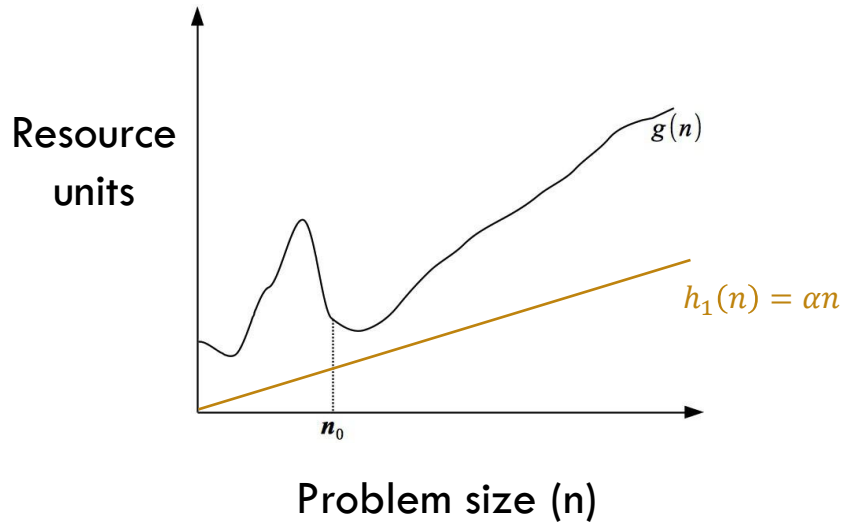
We say that $g(n) \in O(f(n))$ if and only if:

- (1) there exist constants $c > 0$ and $n_0 > 0$ such that for all $n > n_0$ we have that $g(n) \leq cf(n)$
- (2) $f(n)$ is the function that provides the tightest possible upper bound in (1)



Big O notation (example)

Which function would be $f(n)$ in this case?

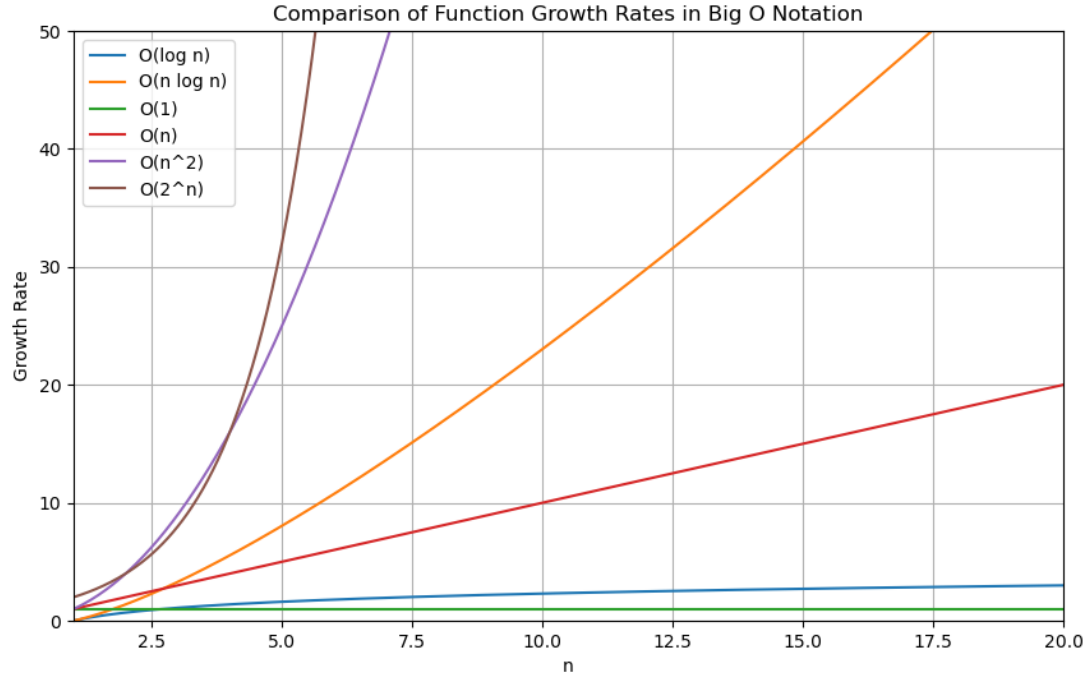


Big O notation (practical considerations)

- > $g(n) \in O(f(n))$ roughly means “ $g(n)$ grows at the rate of $f(n)$ for large enough n ”
- > $f(n)$ is determined by the **leading order** of $g(n)$ (i.e., term that dominates as $n \rightarrow \infty$)
- > **We do not care about constants** when dealing with Big O notation
- > For example:
 - >> $4n^2 - 2n + 1$ is $O(n^2)$ (**quadratic** growth rate)
 - >> $100n$ is $O(n)$ (**linear** growth rate)
 - >> 10^{18} is $O(1)$ (**constant** growth rate, i.e., no growth with n)



Most common rates of growth (i.e., $f(n)$ functions)



Time complexity

- > **Time complexity analysis** determines the rate of growth of the number of **elementary operations** (e.g., comparisons, arithmetic operations, assignment statements, etc.) that an algorithm requires to solve the problem as problem size grows
- > We assume that elementary operations take one unit of time to complete (this is not actually the case when an actual code executes on an actual machine)
- > We are not actually concerned with predicting actual computational times when the algorithm runs in a particular computer, e.g., in microseconds (that would be highly machine-dependent, and much harder to estimate with state-of-the-art hardware)
- > **Warning:** library function calls do not count as elementary operations. They may indeed trigger a complex algorithm for which you need to know the time complexity when determining the time complexity of the calling algorithm
- > Let us start with some simple examples



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;
    for (var i: list) {
        s += i;
    }
    return s;
}
```



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;
    for (var i: list) {
        s += i;
    }
    return s;
}
```



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;           1
    for (var i: list) {    $n$ 
        s += i;
    }
    return s;
}
```



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;           1
    for (var i: list) {    $n$ 
        s += i;           $n$ 
    }
    return s;
}
```



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;           1
    for (var i: list) {    $n$ 
        s += i;            $n$ 
    }
    return s;            1
}
```



Summing elements in a list

Assume that the input list has n elements. At which rate does the number of elementary operations growth with n ? Use Big O notation to express it.

```
int sum(ArrayList<Integer> list)
{
    int s = 0;
    for (var i: list) {
        s += i;
    }
    return s;
}
```

$$g(n) = 2 + 2n \rightarrow g(n) \in O(n)$$

Linear time complexity



Maximum difference among any two elements in a list



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max = 0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max =0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max =0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max = 0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max = 0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
```

```
{
```

```
    int max = 0; 1
```

```
    for (int i=0; i<values.size(); i++) { n
```

```
        for (int j=i+1; j<values.size(); j++) {  $\frac{n(n-1)}{2}$ 
```

```
            int diff = Math.abs(values.get(i)-values.get(j))  $\frac{n(n-1)}{2}$ 
```

```
            if (diff > max) {  $\frac{n(n-1)}{2}$ 
```

```
                max=diff;
```

```
            }
```

```
        }
```

```
    }
```

```
    return max;
```

```
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max = 0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
```

```
{
```

```
    int max = 0; 1
```

```
    for (int i=0; i<values.size(); i++) { n
```

```
        for (int j=i+1; j<values.size(); j++) {  $\frac{n(n-1)}{2}$ 
```

```
            int diff = Math.abs(values.get(i)-values.get(j))  $\frac{n(n-1)}{2}$ 
```

```
            if (diff > max) {  $\frac{n(n-1)}{2}$ 
```

```
                max=diff;  $\frac{n(n-1)}{2}$ 
```

```
            }
```

```
        }
```

```
    }
```

```
    return max; 1
```

```
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max = 0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); j++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$

$$g(n) = 2 + n + 4 \frac{n(n-1)}{2} = 2 + n + 2n^2 - 2n = 2n^2 - n + 2$$

Thus, $g(n) \in O(n^2)$

Quadratic time complexity



Maximum difference among any two elements in a list

Assumption: `values.size()` and `values.get(...)` take unit time

```
int maxDiff(ArrayList<Integer> values)
{
    int max =0;
    for (int i=0; i<values.size(); i++) {
        for (int j=i+1; j<values.size(); i++) {
            int diff = Math.abs(values.get(i)-values.get(j))
            if (diff > max) {
                max=diff;
            }
        }
    }
    return max;
}
```

Note that $\sum_{i=0}^{n-1} i = \frac{n(n-1)}{2}$

$$g(n) = 2 + n + 4 \frac{n(n-1)}{2} = 2 + n + 2n^2 - 2n = 2n^2 - n + 2$$

Thus, $g(n) \in O(n^2)$

Quadratic time complexity

Question: can we do better?



A more interesting example: Find greatest up to

- > Given an **unsorted** list of integers with n elements, find the largest element in the list $\leq x$, with x being a given input integer number. If all the elements in the list are larger than x , return null
- > Let us analyse the time complexity of two different approaches:
 1. Directly search the unsorted list
 2. First sort the list in increasing order, then search the sorted list



Unsorted Greatest Up To



Unsorted Greatest Up To

Integer unsortedGreatestUpTo

```
(ArrayList<Integer> list, int x)
{
    Integer best = null;
    for (var e: list)
    {
        if (e==x)
            return e;
        if ( e<x && (best == null || e>best))
            best=e;
    }
    return best;
}
```



Unsorted Greatest Up To

Integer unsortedGreatestUpTo

(ArrayList<Integer> list, int x)

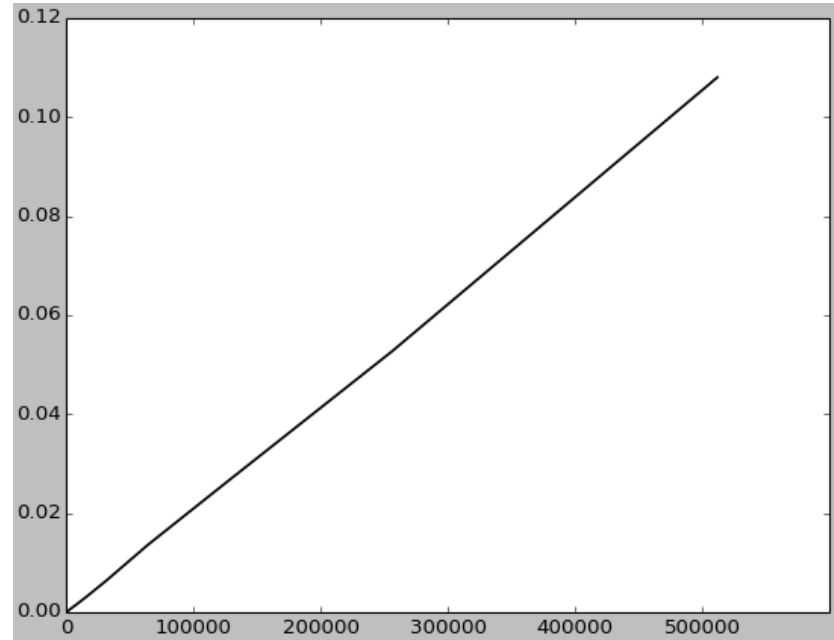
```
{
    Integer best = null;
    for (var e: list)
    {
        if (e==x)
            return e;
        if ( e<x && (best == null || e>best))
            best=e;
    }
    return best;
}
```

Time complexity analysis

- **Best-case:** list.get(0)==x
- **Worst-case:**
 - list == [x-n, x-n+1, ..., x-2, x-1]
 - We perform all comparisons and update best at every loop iteration
 - $g(n) = cn \rightarrow g(n) \in O(n)$
(linear time complexity)



Unsorted Greatest Up To



Actual time measurements versus problem size



Sorted Greatest Up To (Binary search)



Sorted Greatest Up To (Binary search)

Integer sortedGreatestUpTo

```
(ArrayList<Integer> list, int x)
{
    if (list.isEmpty() || list.get(0) > x)
        return null;
    int lower=0;
    int upper=list.size(); //one past the end
    while (upper-lower>1) {
        int mid = (lower+upper)/2; //int division
        if (list.get(mid) <= x)
            lower = mid;
        else
            upper = mid;
    }
    return list.get(lower);
}
```

Note: Loop invariant (assuming no repeated elements)
 $\text{list.get(lower)} \leq x < \text{list.get(upper)}$



Sorted Greatest Up To (Binary search)

Integer sortedGreatestUpTo

```
(ArrayList<Integer> list, int x)
{
    if (list.isEmpty() || list.get(0) > x)
        return null;
    int lower=0;
    int upper=list.size(); //one past the end
    while (upper-lower>1) {
        int mid = (lower+upper)/2; //int division
        if (list.get(mid) <= x)
            lower = mid;
        else
            upper = mid;
    }
    return list.get(lower);
}
```

Time complexity analysis

How many loop iterations?

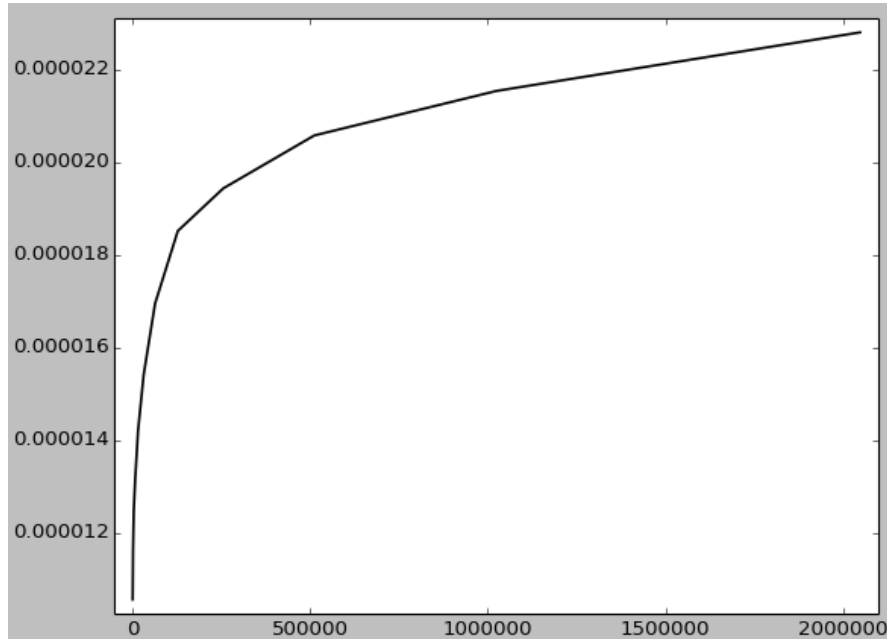
- > Initially, upper-lower==n
- > The difference is halved at every iteration
- > Can halve it at most $\log_2(n)$ times before it becomes 1
- > $g(n) = a \log_2(n) + b \rightarrow g(n) \in O(\log(n))$
(logarithmic time complexity)

Note: Loop invariant (assuming no repeated elements)

$\text{list.get(lower)} \leq x < \text{list.get(upper)}$



Sorted Greatest Up To



Actual time measurements versus problem size



Problem complexity

- > The complexity of a problem is the amount of resources that **any** algorithm that solves the problem **must** use, in the worst case, as a function of the size of the arguments
- > In other words, the complexity of a problem is the **infimum** of the complexities among all algorithms that solve the problem
- > For example, it is possible to proof that **any possible sorting algorithm** that uses pairwise comparisons **needs at least $O(n \log(n))$ comparisons in the worst case**
- > Proving these kind of results is out of the scope of the course, and in general, it requires advanced arguments in the mathematical theory of computation



<Exercises>

Which of the following statements about algorithm complexity are correct? (Select one or more)

- A. Studying the time complexity of an algorithm amounts to modelling how much time it spends when run on a particular computer
- B. We use Big O notation to express the time and memory complexity of an algorithm
- C. An algorithm with two nested loops always has quadratic time complexity
- D. An algorithm that performs a vast amount of operations independently of problem size has exponential time complexity
- E. If an algorithm performs 8 times as many operations when we double problem size, it has $O(n^2)$ time complexity (i.e., quadratic complexity)



<Exercises>

Assume that, in the code below, `list` is of type `LinkedList<Integer>`. Which is the worst-case time complexity of the code in big O notation, with n denoting the number of elements in the list? What about best-case time complexity?

```
sum = 0
for (int i = 0; i < list.size(); i++)
    sum += list.get(i)
```

- A. $O(n^2)$
- B. $O(\log(n))$
- C. $O(n \log(n))$
- D. $O(n)$
- E. $O(n^3)$



<Exercises>

For more exercises on time complexity, please refer to the PDF document accompanying these slides on Canvas



Thank you



Australian
National
University

TEQSA PROVIDER ID: PRV12002 (AUSTRALIAN UNIVERSITY)
CRICOS PROVIDER CODE: 00120C